

Aufbau

1. Motivation
2. Formalisierung erarbeiten
3. Soundness & Completeness Proofs (Sketch?)
4. Deadlock Algorithm

Keywords

- **Message Passing:** Query, Response, Cast
- **Remote Procedure Calls**
- **Deadlock:** Cyclic wait
- **Monitors:** Analyse traffic
 - Probes: Communication with other monitors
- **Black-Box:** No introspection

Quirks

- The monitored path might be different!

Proofs and Definitions

- Criterion 2.1 (Monitor Instrumentation Transparency)
 - Operational Correspondance vs. Simulation Relation
 - soundness allows some extra steps
- Criterion 2.2 (Preciseness of Deadlock Detection)
 - completeness allows some extra steps
- Lock
- Deadlock
- Deadlockset
- Lemma 3.8 (Persistence of deadlocks).

Todo

- Probing
- Process Calculi?
- SRPC relation to RPC?

Correct Black-Box Monitors for Distributed Deadlock Detection: Formalisation and Implementation

Disecting The Title

- We are given a network setting with communicating RPC Services
- **RPC**: Remote Procedure Call (Query, Response, Cast)
- **Deadlock**: Cyclic waiting for resources in such a network
- **Distributed**: No central authority (due to performance)
- **Blackbox**: No introspection of services, only communication behaviour
- **Monitors**: Wrappers around service that observe communication

Contributions

- Formal model for RPC services
- Formalised method for distributed, black-box, outline instrumentation
- Monitoring algorithm that detects deadlock in a sound and complete way
- Implementation of DDMon (for Erlang/OTP)

Agenda

1. Formalising Processes & Services
2. Formalising Networks
3. Installing Monitors
4. Proof for Deadlock-detection
5. Deadlock Detection Algorithm
6. Proof of Soundness & Completeness

Wanted Properties

1. **Distributed**: No centralized component
2. **Blackbox and outline**: Detect deadlocks by just observing the incoming and outgoing messages of the service they monitor
3. **Transparent**: Don't alter the execution
4. **Precise**: Detect Deadlock iff it existst

Nodes

Formalism: Services

- The model is reduced to *Single-threaded Remote Procedure Calls* (SRPC)

Communication tag $t := Q \mid R \mid C$

Name $n := n_0 \mid n_1 \mid \dots$

Client $n^\perp := n \mid \perp$

Process $P := \dots$

Queue $q := \varepsilon \mid n(t) \mid q \# q$

Service $S := \langle q^i \mid P \mid q^o \rangle$

- The Behaviour of P must be compatible with an **abstract SRPC Process G**
- TODO: underline lines?

Formalism: LTS Semantic of abstract SRPC Process G

> Simulate a single Process P

$$G := \text{Ready} \mid \text{Working}(n_c^\perp) \mid \text{Locked}(n_c^\perp, n_s)$$

$$\gamma := ?n(t) \mid !n(t) \mid \tau$$

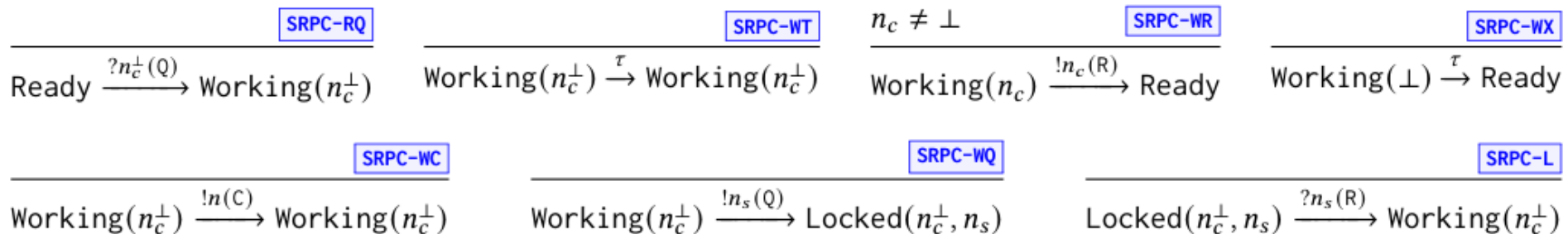


Figure 1: LTS semantics of the abstract SRPC process G

Formalism: LTS Semantic of Services

> Extend a process with queues (and search operations)

$$\delta := ![n(t)]$$

$$\beta := ? n(t) \mid ![n(t)] \mid \tau(? n(t)) \mid \tau(!n(t)) \mid \tau(\tau)$$

$$\begin{array}{c}
\frac{}{n(t) ++ q \xrightarrow{![n(t)]} q} \quad \text{QUE-FIND} \qquad \frac{q_1^i = n(t) ++ q_0^i}{\langle q_0^i | P | q^o \rangle \xrightarrow{?n(t)} \langle q_1^i | P | q^o \rangle} \quad \text{SER-I} \qquad \frac{q_0^o = n(t) ++ q_1^o}{\langle q^i | P | q_0^o \rangle \xrightarrow{!n(t)} \langle q^i | P | q_1^o \rangle} \quad \text{SER-O} \qquad \frac{P_0 \xrightarrow{\tau} P_1}{\langle q^i | P_0 | q^o \rangle \xrightarrow{\tau(\tau)} \langle q^i | P_1 | q^o \rangle} \quad \text{SER-TT} \\
\\
\frac{n(t) \neq n'(t') \quad q \xrightarrow{![n'(t')]} q'}{n(t) ++ q \xrightarrow{![n'(t')]} n(t) ++ q'} \quad \text{QUE-SEEK} \qquad \frac{q_0^i \xrightarrow{![n(t)]} q_1^i \quad P_0 \xrightarrow{?n(t)} P_1}{\langle q_0^i | P_0 | q^o \rangle \xrightarrow{\tau(?n(t))} \langle q_1^i | P_1 | q^o \rangle} \quad \text{SER-TI} \qquad \frac{P_0 \xrightarrow{!n(t)} P_1 \quad q_1^o = n(t) ++ q_0^o}{\langle q^i | P_0 | q_0^o \rangle \xrightarrow{\tau(!n(t))} \langle q^i | P_1 | q_1^o \rangle} \quad \text{SER-TO}
\end{array}$$

Figure 2: LTS semantics of services

Networks

Formalism: Network

$$\alpha := \tau(\gamma @ n) \mid n_0 - (t) \rightarrow n_1$$

$$\frac{N(n) \xrightarrow{\tau(\gamma)} S \quad \boxed{\text{NET-TAU}}}{N \xrightarrow{\tau(\gamma @ n)} N[n \leftarrow S]}$$

$$\frac{N(n_0) \xrightarrow{!n_1(C)} S_0 \quad N[n_0 \leftarrow S_0](n_1) \xrightarrow{?\perp(Q)} S_1}{N \xrightarrow{n_0 - (C) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1] \quad \boxed{\text{NET-CAST}}}$$

$$\frac{t \neq C \quad N(n_0) \xrightarrow{!n_1(t)} S_0 \quad N[n_0 \leftarrow S_0](n_1) \xrightarrow{?n_0(t)} S_1}{N \xrightarrow{n_0 - (t) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1] \quad \boxed{\text{NET-COM}}}$$

Figure 3: LTS semantics of services

Formalism: Deadlocks Set

- Todo: Deadlock of single node
- Todo: Deadlock set

Monitors

Formalism: Monitors

$$\hat{m} := ? n(t) \mid !n(t) \mid ? n(\hat{p}) \mid !n(\hat{p})$$

$$\hat{q} := \varepsilon \mid \hat{m} \mid \hat{q} + \hat{q}$$

$$\hat{S} := \langle \hat{q} \mid P \mid S \rangle$$

Monitored service action:

$$\hat{\beta} := \beta \mid ? n(\hat{p}) \mid !n(\hat{p}) \mid \hat{\tau}(? n(t)) \mid \hat{\tau}(!n(t)) \mid \hat{\tau}(? n(\hat{p}))$$

- $\hat{p}$ is a probe
- A monitor $\hat{M}$

Monitors

- A monitored Service: $\langle \hat{q} \mid \hat{M} \mid S \rangle$

Monitor algorithm function: $\hat{\mathcal{A}}: (\mathcal{M})x\hat{m} \rightarrow \hat{\mathcal{M}} \times \hat{q}$

Formalism: Monitored Network action

$$\hat{\alpha} := \alpha \mid \hat{\tau}(\gamma @ n) \mid n_0 - (\hat{p}) \rightarrow n_1$$

monitored path: $\hat{\sigma}$

$$\begin{array}{c}
\frac{\hat{N}(n_0) \xrightarrow{!n_1(t)} \hat{S}_0 \quad \hat{N}[n_0 \leftarrow \hat{S}_0](n_1) \xrightarrow{?n_0(t)} \hat{S}_1 \quad t \neq \mathsf{C}}{N \xrightarrow{n_0-(t) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]} \text{MNET-SCOM}
\end{array}
\quad
\frac{\hat{N}(n_0) \xrightarrow{!n_1(\mathsf{C})} \hat{S}_0 \quad \hat{N}[n_0 \leftarrow \hat{S}_0](n_1) \xrightarrow{?\perp(Q)} \hat{S}_1}{N \xrightarrow{n_0-(t) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]} \text{MNET-SCAST}$$

$$\frac{\hat{N}(n_0) \xrightarrow{!n_1(\hat{p})} \hat{S}_0 \quad \hat{N}[n_0 \leftarrow \hat{S}_0](n_1) \xrightarrow{?n_0(\hat{p})} \hat{S}_1}{\hat{N} \xrightarrow{n_0-(\hat{p}) \rightarrow n_1} \hat{N}[n_0 \leftarrow \hat{S}_0][n_1 \leftarrow \hat{S}_1]} \text{MNET-COM}$$

$$\frac{\hat{N}(n) \xrightarrow{\tau(\gamma)} \hat{S}}{\hat{N} \xrightarrow{\tau(\gamma @ n)} \hat{N}[n \leftarrow \hat{S}]} \text{MNET-STAU}$$

$$\frac{\hat{N}(n) \xrightarrow{\hat{\tau}(\hat{\beta})} \hat{S}}{\hat{N} \xrightarrow{\hat{\tau}(\hat{\beta} @ n)} \hat{N}[n \leftarrow \hat{S}]} \text{MNET-TAU}$$

Figure 4: LTS semantics of monitored networks

Instrumentation

- Monitor instrumentation: $\mathcal{M} : \mathcal{N} \rightarrow \widehat{\mathcal{N}}$
- Deinstrumentation: $\mathcal{M}^{(-1)}$
- Monitor stripping operation:

$$\hat{S} \downarrow: \langle \hat{q} \mid \widehat{M} \mid \langle q^i \mid P \mid q^o \rangle \rangle = \\ \langle q^i + +\text{filter}^{i(\hat{q})} \mid P \mid q^o + +\text{filter}^{o(\hat{q})} + +q^o \rangle$$

Stripping of monitor path: $\text{hat}(\sigma) \downarrow: \dots$

Theorem 4.8 (Black-box instrumentation transparency — completeness).

The Algorithm

- Monitor states $\hat{M}$ is a record with the fields “probe” | “waiting” | “alarm”
- Implementation of $\hat{\mathcal{A}}$

Wellformedness of SRPC

- Definition: Initial Network
- Definition: Client
- Definition: SRPC well-formedness
- Definition: Wellformed SRPC Network
- Theorem: Wellformedness as an *invariant*
- Definition: Complete Lock Knowledge
- Definition: Alarm Condition
- Definition: Sound lock knowledge
- Theorem: Sound lock knowledge is an *invariant*
- Theorem 6.13 (Deadlock detection preciseness)

- Theorem 6.13 (eventual deadlock reporting)

Evaluation

- Overhead is neglectible